



PES

UNIVERSITY

CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

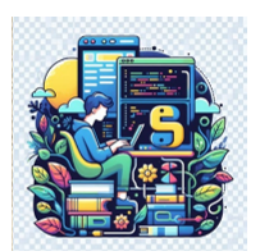
PROGRAMMING WITH PYTHON

Object-Oriented Programming (OOP) and Advanced Concepts

Inheritance

Lekha A

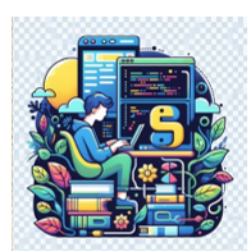
Computer Applications



PROGRAMMING WITH PYTHON

Inheritance

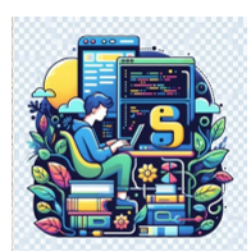
- When one class inherits from another, it automatically takes on all the attributes and methods of the first class.
- The original class is called the **parent class**, and the new class is the **child class**.
- The child class inherits every attribute and method from its parent class but is also free to define new attributes and methods of its own.



PROGRAMMING WITH PYTHON

`__init__()` Method for a Child Class

- When creating an instance from a child class is to assign values to all attributes in the parent class.
- To do this, the `__init__()` method for a child class needs help from its parent class.



PROGRAMMING WITH PYTHON

__init__() Method for a Child Class

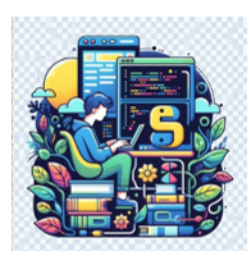
```
class Car(): 1
    """A simple attempt to represent a car."""
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
        self.odometer_reading = 0

    def get_descriptive_name(self):
        long_name = str(self.year) + ' ' + self.make + ' ' + self.model
        return long_name

    def read_odometer(self):
        print("This car has " + str(self.odometer_reading) + " kms on it.")

    def update_odometer(self, mileage):
        if mileage >= self.odometer_reading:
            self.odometer_reading = mileage
        else:
            print("You can't roll back an odometer!")

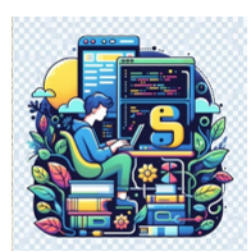
    def increment_odometer(self, kms):
        self.odometer_reading += kms
```



PROGRAMMING WITH PYTHON

`__init__()` Method for a Child Class

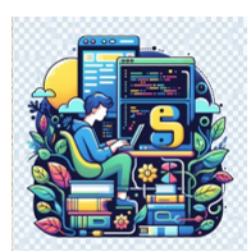
- At **1** start with Car.
- When a child class is created, the parent class must be part of the current file and must appear before the child class in the file.



PROGRAMMING WITH PYTHON

`__init__()` Method for a Child Class

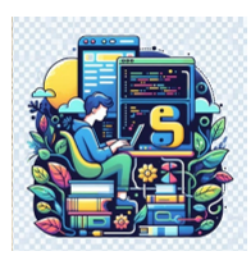
```
class GamingLaptop(Laptop):  
    """Represent aspects of a laptop specific to gaming laptops."""  
    def __init__(self, brand, model, price, gpu):  
        """Initialize attributes of the parent class."""  
        super().__init__(brand, model, price)  
        self.gpu = gpu
```



PROGRAMMING WITH PYTHON

`__init__()` Method for a Child Class

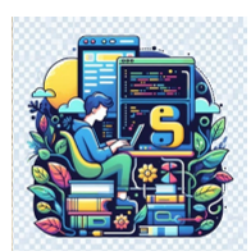
- At **2** define the child class, GamingLaptop.
- The name of the parent class must be included in parentheses in the definition of the child class.



PROGRAMMING WITH PYTHON

`__init__()` Method for a Child Class

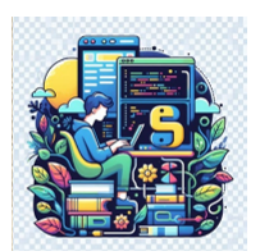
- The `__init__()` method at **3** takes in the information required to make a Laptop instance.
- The `super()` function at **4** is a special function that helps Python make connections between the parent and child class.



PROGRAMMING WITH PYTHON

`__init__()` Method for a Child Class

- This line tells Python to call the `__init__()` method from GamingLaptop's parent class, which gives a GamingLaptop instance all the attributes of its parent class.
- The name **super** comes from a convention of calling the parent class a superclass and the child class a subclass.



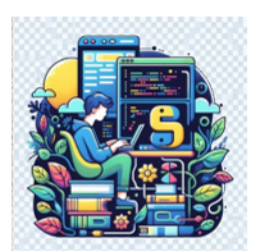
PROGRAMMING WITH PYTHON

__init__() Method for a Child Class

```
gaming_laptop = GamingLaptop("Asus", "ROG Strix", 150000, "NVIDIA RTX 4070")
```

5

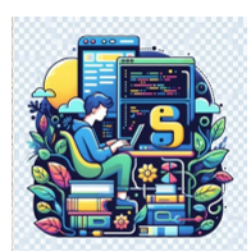
- At **5** make an instance of the GamingLaptop class, and store it in gaming_laptop.
- This line calls the __init__() method defined in GamingLaptop, which in turn tells Python to call the __init__() method defined in the parent class Laptop.



PROGRAMMING WITH PYTHON

Defining Attributes and Methods for the Child Class

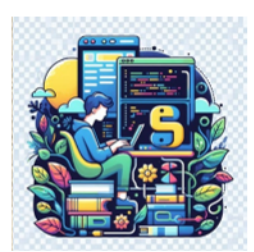
- Can add any new attributes and methods necessary to differentiate the child class from the parent class.
- Example
 - Add an attribute that's specific to gaming laptop(a gpu) and a method to report on this attribute.



PROGRAMMING WITH PYTHON

Defining Attributes and Methods for the Child Class

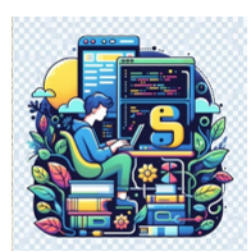
```
class GamingLaptop(Laptop):  
    """Represent aspects of a laptop specific to gaming laptops."""  
    def __init__(self, brand, model, price, gpu):  
        """Initialize attributes of the parent class."""  
        super().__init__(brand, model, price)  
        1 self.gpu = gpu  
  
    2 def boost_mode(self):  
        """Simulate enabling gaming boost mode."""  
        print(f"{self.brand} {self.model} is running in Boost Mode with {self.gpu} GPU!")
```



PROGRAMMING WITH PYTHON

Defining Attributes and Methods for the Child Class

- At **1** add a new attribute `self.gpu`.
- This attribute will be associated with all instances created from the `GamingLaptop` class but won't be associated with any instances of `Laptop`.
- Add a method called `boost_mode()` that prints information about the gpu at **2**.



PROGRAMMING WITH PYTHON

Defining Attributes and Methods for the Child Class

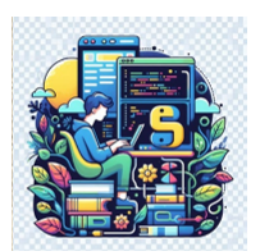
```
print(gaming_laptop.get_descriptive_info())  
gaming_laptop.check_battery()  
gaming_laptop.boost_mode()
```

--- Gaming Laptop Details ---

Asus Rog Strix Priced At ₹150000

This laptop has 100% battery remaining.

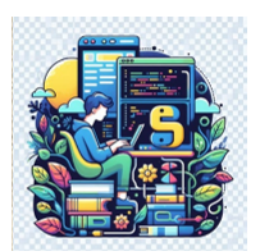
Asus ROG Strix is running in Boost Mode with NVIDIA RTX 4070 GPU!



PROGRAMMING WITH PYTHON

Overriding Methods from the Parent Class

- Any method from the parent class can be overridden that doesn't fit what with the child class.
- To do this, define a method in the child class with the same name as the method that needs to be overridden in the parent class.
- Python will disregard the parent class method and only pay attention to the method defined in the child class.



PROGRAMMING WITH PYTHON

Overriding Methods from the Parent Class

- In the class Laptop

```
def turn_on(self):  
    print(f"{self.brand} {self.model} is now turned ON.")
```

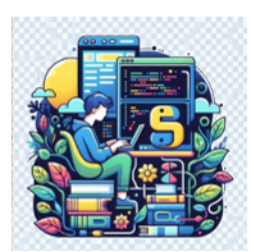
- In the class GamingLaptop

```
def turn_on(self):  
    return f"{self.brand} {self.model} is starting in high-performance gaming mode with {self.gpu}!"
```

`laptop1.turn_on()` HP Pavilion is now turned ON.

`gaming_laptop.turn_on()`

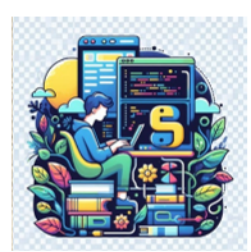
'Asus ROG Strix is starting in high-performance gaming mode with NVIDIA RTX 4070!'



PROGRAMMING WITH PYTHON

Uses of Inheritance

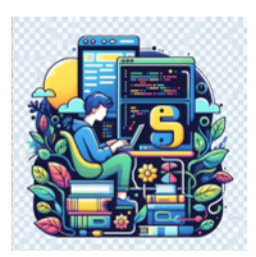
- Since a child class can inherit all the functionalities of the parent's class, this allows code reusability.
- Once a functionality is developed, it can be simply inherited.
 - No need to reinvent the wheel.
 - This allows for cleaner code and easier to maintain.



PROGRAMMING WITH PYTHON

Uses of Inheritance

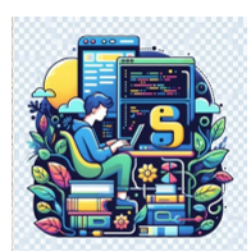
- Since functionalities can be added in the child class, only the useful functionalities can be inherited and other required features can be defined.



PROGRAMMING WITH PYTHON

Types of Inheritance

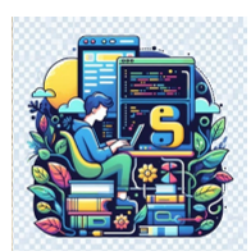
- Multi level inheritance
 - It can be done to any depth
- Multiple inheritance
 - It can be done.
 - MRO is done by Python



PROGRAMMING WITH PYTHON

Method Resolution Order

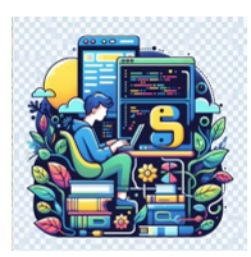
- **MRO** stands for **Method Resolution Order** in Python.
- It is the order in which Python looks for a method or attribute in a hierarchy of classes when it is invoked.



PROGRAMMING WITH PYTHON

Method Resolution Order

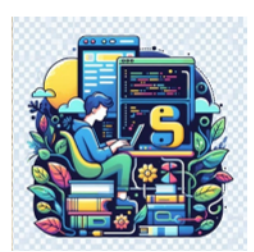
- MRO provides a linear order of inheritance, even in cases of multiple inheritance, ensuring that each class in the hierarchy is considered exactly once.
- Python uses the C3 Linearization Algorithm (also called C3 superclass linearization) to compute the MRO.
- This algorithm respects the order of base classes in the class definition and ensures consistency in method resolution.



PROGRAMMING WITH PYTHON

Method Resolution Order

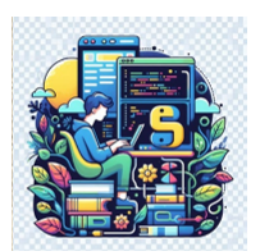
- Use the `ClassName.mro()` method or the `help(ClassName)` function to see the MRO or `__mro__` attribute of a class.



PROGRAMMING WITH PYTHON

Usage of Method Resolution Order

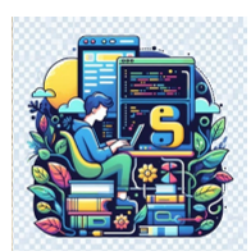
- Single inheritance
 - The MRO is straightforward—Python looks in the current class and its single parent class.
- Multiple inheritance
 - Python determines a consistent order to resolve methods by following the C3 Linearization rules.



PROGRAMMING WITH PYTHON

Usage of Method Resolution Order

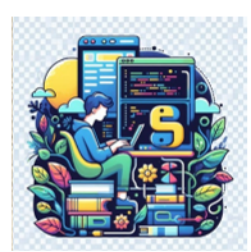
- The diamond problem occurs when a class inherits from two classes that both inherit from a common base class.
- Example
 - Python resolves the method in B before C due to the MRO order computed by the C3 algorithm.



PROGRAMMING WITH PYTHON

Rules of C3 Linearization

- Children precede their parents.
- Parents are considered in the order they are listed in the base class declaration.
- A class is considered only after all its parent classes have been considered.



PROGRAMMING WITH PYTHON

Method Resolution Order

Base Class

```
class Root:  
    def do(self):  
        print("Root.do (end of chain)")
```

Inherited Classes

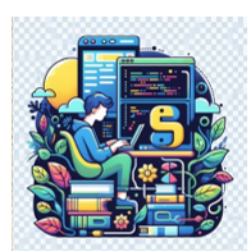
```
class A(Root):  
    def do(self):  
        print("A.do")  
        super().do()
```

```
class B(A):  
    def do(self):  
        print("B.do")  
        super().do()
```

```
class C(A):  
    def do(self):  
        print("C.do")  
        super().do()
```

```
class D(B, C):  
    """D(B, C): MRO will be D -> B -> C -> A -> Root -> object"""  
    pass
```

```
class E(C, B):  
    """E(C, B): MRO will be E -> C -> B -> A -> Root -> object"""  
    pass
```



PROGRAMMING WITH PYTHON

Method Resolution Order

```
print("MRO of D:", [cls.__name__ for cls in D.mro()])  
print("Calling D().do():")  
D().do()
```

MRO of D: ['D', 'B', 'C', 'A', 'Root', 'object']

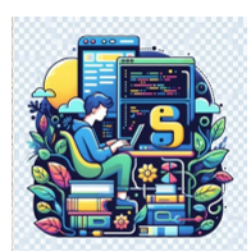
Calling D().do():

B.do

C.do

A.do

Root.do (end of chain)



PROGRAMMING WITH PYTHON

Method Resolution Order

```
print("\nMRO of E:", [cls.__name__ for cls in E.mro()])  
print("Calling E().do():")  
E().do()
```

MRO of E: ['E', 'C', 'B', 'A', 'Root', 'object']

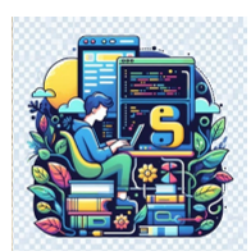
Calling E().do():

C.do

B.do

A.do

Root.do (end of chain)



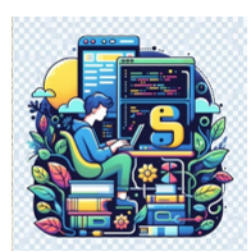
PROGRAMMING WITH PYTHON

Method Resolution Order

```
print(D.__mro__)  
print(E.__mro__)
```



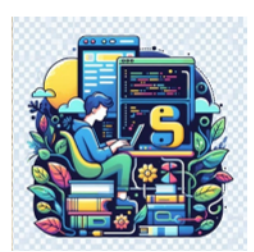
```
(<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <  
class '__main__.Root'>, <class 'object'>)  
(<class '__main__.E'>, <class '__main__.C'>, <class '__main__.B'>, <class '__main__.A'>, <  
class '__main__.Root'>, <class 'object'>)
```



PROGRAMMING WITH PYTHON

Advantages of MRO

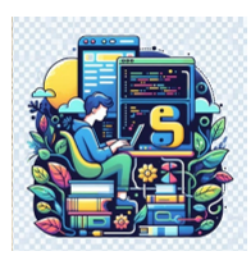
- Avoids Ambiguity
 - Ensures Python knows which method to invoke, even in complex inheritance hierarchies.
- Ensures Consistency
 - Guarantees a predictable and consistent method lookup order.



PROGRAMMING WITH PYTHON

Recap

- Inheritance
- `__init__()` Method for a Child Class
- Defining Attributes and Methods for the Child Class
- Overriding Methods from the Parent Class
- Uses of Inheritance



PROGRAMMING WITH PYTHON

Recap

- Types of Inheritance
- Method Resolution Order



THANK YOU

Lekha A
Computer Applications